Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

La Standardisation et la Normalisation : Différences Théoriques et Utilisations Pratiques

Standardisation :

Théorie :

But : Transformer les données pour avoir une moyenne de 0 et un écart-type de 1.

Effet : Les données standardisées ont une variance unitaire et peuvent prendre n'importe quelle valeur. (positive et négatif)

Utilisations Pratiques :

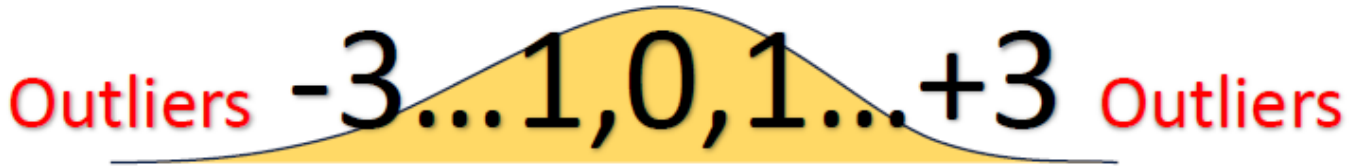
1. Détection des Valeurs Aberrantes : Utilisation des z-scores pour identifier les outliers (valeurs au-delà de ±3).

2. Algorithmes de Machine Learning : Régression Linéaire et Logistique : Meilleure performance avec des données standardisées.

Analyse en Composantes Principales (ACP) : Requiert des caractéristiques sur la même échelle.

Clustering K-Means : Fonctionne mieux avec des caractéristiques standardisées pour éviter la domination de certaines caractéristiques.

• Après l'application de la Standardisation sur vous données normales, votre distribution sera comme ça :



Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

La Standardisation et la Normalisation : Différences Théoriques et Utilisations Pratiques

Normalisation :

Théorie :

But : Échelonner les données pour qu'elles s'inscrivent dans une plage spécifique, généralement [0, 1].
Effet : Les données normalisées sont limitées à une plage fixe sans changer la forme de la distribution.
Utilisations Pratiques :

Réseaux de Neurones :Convergence plus rapide avec des entrées normalisées.
K-Nearest Neighbors (KNN) :Distance calculée de manière équitable entre les caractéristiques.
Visualisation des Données :Améliore l'interprétabilité des graphiques en maintenant une plage cohérente.
• Après l'application de la Normalisation vous données ressembleront à ceci :



Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

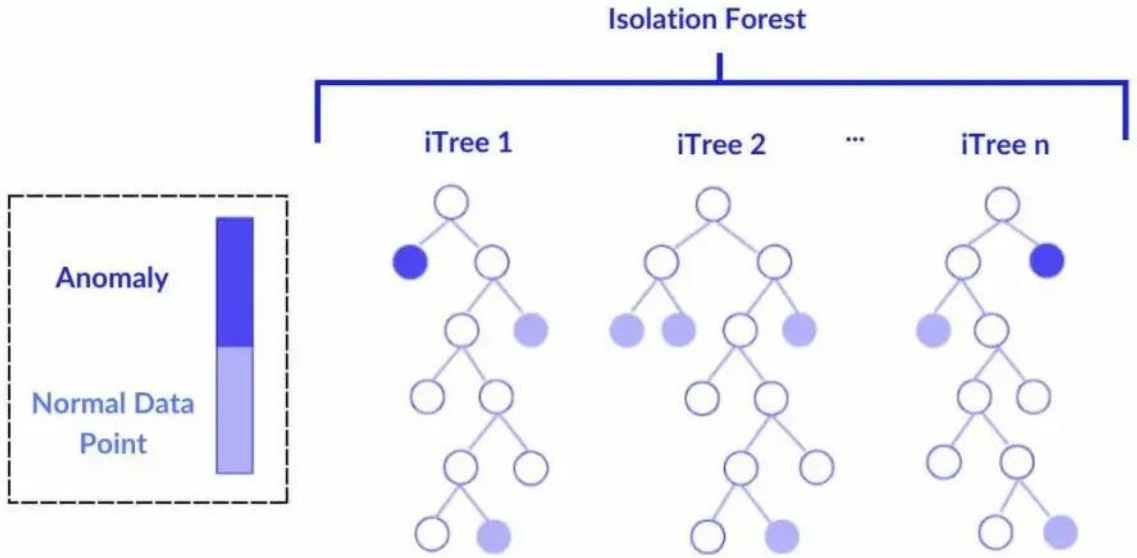
Identification par les techniques statistiques avancées : Isolation Forest (Intrusion detection algorithm)

Qu'est-ce que l'Isolation Forest?

L'Isolation Forest (Forêt d'Isolation) est un algorithme de détection d'anomalies basé sur le concept que les anomalies sont "rares et différentes". Contrairement aux méthodes traditionnelles qui modélisent le comportement normal, l'Isolation Forest isole directement les anomalies.

Fonctionnement de l'Isolation Forest :

L'algorithme fonctionne en isolant rapidement les anomalies dans des arbres de décision. Les anomalies nécessitent moins de divisions pour être isolées, car elles ont des caractéristiques distinctes qui les rendent plus faciles à séparer des valeurs normales.



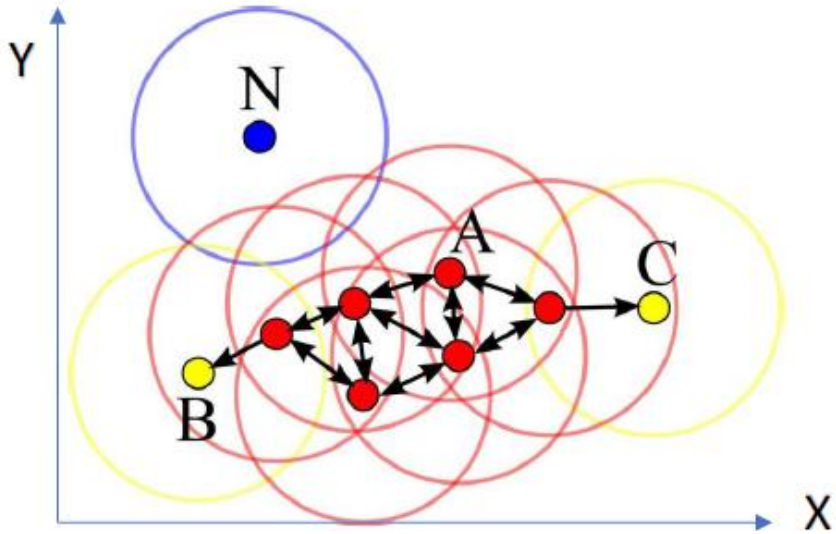
Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

Techniques statistiques avancées : DBSCAN Algorithm

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) est un algorithme de clustering basé sur la densité. Il identifie les zones denses de points de données en se basant sur le nombre de points voisins « Neaboars » dans une région donnée. Les points qui ne sont pas atteints par suffisamment de voisins sont considérés comme du bruit. DBSCAN est efficace pour détecter des groupes de formes arbitraires et s'adapte bien à différentes densités locales dans l'ensemble de données.




```
A : Core points (points centraux)
B, C : Density-connected (connectés par densité)
N : Noise (bruit) => Outliere
```

REMARQUE:

DBSCAN est utilisé dans **des situations complexes** où plusieurs colonnes ou dimensions sont prises en compte simultanément pour construire de nombreux clusters. Cette approche diffère des méthodes précédentes qui détectent principalement les valeurs aberrantes en se basant sur les données d'une seule une dimension.

DBSCAN est un algorithme de regroupement dont le but principal est d'identifier les régions denses dans les données. Bien qu'il puisse être utilisé pour détecter les valeurs aberrantes (Outliers).

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

Gestion et traitement des Outliers : Consielles

• Gérer les outliers est essentiel pour garantir la précision des analyses de données. Pour ce faire, diverses méthodes peuvent être utilisées, telles que la règle de IQR, les algorithmes DBSCAN et Isolation Forest.

• Les outliers ont une signification statistique, pouvant signaler des erreurs ou des événements spéciaux dans les données. Leur présence peut influer considérablement sur les résultats et les conclusions tirées des analyses.

• Les Z-scores sont souvent utilisés pour identifier ces valeurs aberrantes.

• Enfin, la gestion des outliers est un processus continu qui nécessite une surveillance constante pour maintenir la qualité des analyses et des résultats.

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

Identification par les techniques statistiques avancées : DBSCAN multi-clustering

Application pratique avec Python

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Pour cette illustration de DBSCAN, voici quelques points importants :

1. Paramètres Importants :

eps (epsilon) : Rayon maximal pour qu'un point soit considéré comme voisin.

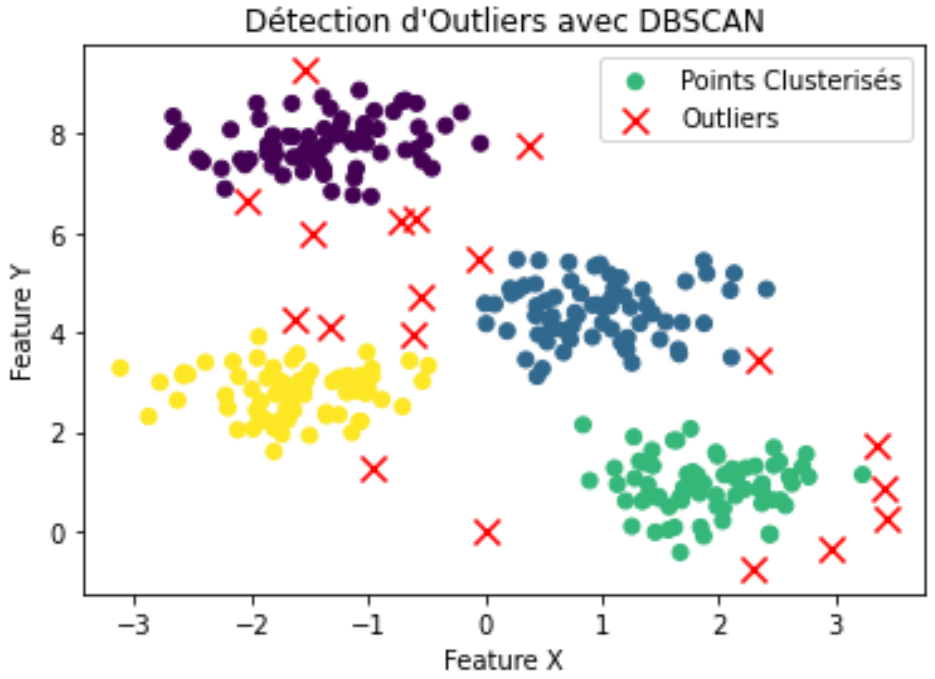
min_samples : Nombre minimal de points dans un cluster.

2. Commandes Essentielles en Python :

- **DBSCAN(eps=0.5, min_samples=5)** # Initialisation de l'algorithme DBSCAN
- **fit_predict(data)** # Prédire les clusters
- **clustered_data = data [labels != -1]** # Séparation des données clusterisées
- **outlier_data = data [labels == -1]** # Séparation des outliers fonction des labels

3. Visualisation avec Matplotlib :

- **Scatter plot** avec des points clusterisés colorés selon les clusters identifiés.
- **Outliers marqués en rouge** avec le symbole ***'x'.***
- Affichage du titre et des axes.



Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

Identification par les techniques statistiques avancées : DBSCAN multi-clustering

Application pratique avec Python

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Importer les bibliothèques et générer une distribution de données :

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_blobs

# Générer des données de test avec quelques clusters et outliers
data, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)
outlier = np.array([[0, 0]])
data_with_outlier = np.concatenate([data, outlier])
```

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

Identification par les techniques statistiques avancées : DBSCAN multi-clustering

Application pratique avec Python

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Plus sur la génération de distribution de données avec make_blobs:

make_blobs est une fonction de la bibliothèque sklearn.datasets en Python, utilisée pour générer des ensembles de données synthétiques pour des exemples et des tests. Plus précisément, elle crée des blobs de points de données, ce qui est utile pour les démonstrations de techniques de clustering ou d'autres algorithmes de machine Learning.

Voici une brève description de ses principaux paramètres :

n_samples : Nombre total de points générés. Vous pouvez également passer une liste pour spécifier le nombre de points pour chaque cluster.

cluster.centers : Nombre de centres de clusters à générer, ou les coordonnées des centres de clusters.

n_features : Nombre de caractéristiques pour chaque point.

cluster_std : L'écart type des clusters.

center_box : Les limites des valeurs pour les centres des clusters.

shuffle : Indique si les échantillons doivent être mélangés.

random_state : Contrôle le générateur de nombres aléatoires pour la reproductibilité des résultats.

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

Identification par les techniques statistiques avancées : DBSCAN multi-clustering

Application pratique avec Python

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

C'est notre section dédiée de l'application pour utiliser DBSCAN dans la détection des valeurs aberrantes. Il suffit d'appliquer cette méthode pour identifier les outliers :

Application de DBSCAN : Le modèle DBSCAN est configuré avec des paramètres spécifiques (eps=0.5 et min_samples=5) et appliqué aux données contenant des outliers.

```
# Appliquer DBSCAN avec des paramètres spécifiques
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(data_with_outlier)

# Séparer les données clusterisées et les outliers
clustered_data = data_with_outlier[labels != -1]
outlier_data = data_with_outlier[labels == -1]
```

Séparation des données : Les données sont ensuite divisées en deux groupes : les données clusterisées (celles qui ne sont pas des outliers) et les outliers (données avec le label -1).

Approfondir les Systèmes Décisionnels

Approfondir les statistiques :

Maitriser les mesures de dispersion et de position :

Identification par les techniques statistiques avancées : DBSCAN multi-clustering

Application pratique avec Python

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Solution de la Visualisation avec Matplotlib :

Signifie que la colormap "viridis" est utilisée pour colorer les points clusterisés.

Spécifie que les couleurs des points sont déterminées par les labels des clusters, en excluant les outliers (qui ont un label de -1)

spécifient les coordonnées des points sur les axes X et Y respectivement.

```
# Afficher les résultats avec un scatter plot
plt.scatter(clustered_data[:, 0], clustered_data[:, 1], c=labels[labels != -1], cmap='viridis',
label='Points Clusterisés')
plt.scatter(outlier_data[:, 0], outlier_data[:, 1], c='red', marker='x', s=100, label='Outliers')

plt.title('Détection d\'Outliers avec DBSCAN')
plt.xlabel('Feature X')
plt.ylabel('Feature Y')
plt.legend()
plt.show()
```

La fonction plt.scatter() attend les coordonnées X et Y comme ses premiers deux arguments positionnels, et non comme des arguments nommés X et Y.

Ce script visualise la séparation des points en clusters et les outliers détectés par l'algorithme DBSCAN, en utilisant deux caractéristiques des données pour les axes X et Y.